Gujarat Police Hackathon

Docker Setup, Containerization, Commands & Troubleshooting

Document series: Part 2 - Docker. This document connects with the separate MediaMTX + FFmpeg document by documenting the container layer used to run the streaming system.

Scope: This document records how the FastAPI backend, MySQL database and MediaMTX were containerized; how Docker networking and ports work; the Dockerfiles and Compose configuration used; the problems encountered; and the fixes that made the system work.

1. What We Wanted Docker to Solve

The project originally depended on software installed directly on the Windows machine. The objective of Dockerization was to move the application's runtime dependencies into reproducible containers so the project can be deployed on another machine without rebuilding the whole environment manually.

```
Before:
Windows
  + Python virtual environment
  + MySQL installed locally
  + MediaMTX executable
  + FFmpeg executable

After this Docker stage:
Docker Compose
  + Backend container
  + MySQL container
  + MediaMTX container
```

The streaming-specific MediaMTX and FFmpeg work is documented separately. This document focuses on the container layer that makes those components portable.

2. Technologies Used in the Docker Stage

Technology	Why it was used
Docker	Runs isolated application services in containers.
Docker Compose	Defines, builds, networks and starts the multiple containers together.
Python 3.12	Runtime used by the FastAPI backend image.
FastAPI / Uvicorn	Backend REST API running inside the backend container.
MySQL 8.0	Database container storing vendor and camera records.
MediaMTX 1.21.0	Streaming container connected to the real camera RTSP source.
YAML	Compose and MediaMTX configuration format.
PowerShell	Windows shell used to operate Docker, inspect containers and test APIs.

3. Project Structure for Docker

```
GUJARAT_POLICE_HACKETHON/
|
+-- backend/
|   +-- app/
|   +-- requirements.txt
|
+-- database/
|   +-- init.sql
|
+-- deployment/
|   +-- docker-compose.yml
|   +-- docker/
|       +-- backend.Dockerfile
|       +-- mediamtx.Dockerfile
|
```

```
+-- streaming/
|   +-- mediamtx_v1.21.0_windows_amd64/
|       +-- mediamtx.exe        (Windows backup/local file)
|       +-- mediamtx.yml        (working configuration)
|
+-- .gitignore
```

The Windows MediaMTX executable and local FFmpeg binaries were kept locally for testing/backups, but the portable Docker setup uses the appropriate Linux MediaMTX binary inside its container.

4. Backend Dockerfile

The backend image provides Python, installs the application's requirements, copies the backend source, installs FFmpeg inside the Linux image where needed, and starts Uvicorn.

```
FROM python:3.12-slim

WORKDIR /app

COPY backend/requirements.txt ./requirements.txt

RUN pip install --no-cache-dir -r requirements.txt

COPY backend ./backend

RUN apt-get update \
    && apt-get install -y ffmpeg \
    && rm -rf /var/lib/apt/lists/*

EXPOSE 8000

CMD ["uvicorn", "backend.app.main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

Why EXPOSE 8000? FastAPI listens on port 8000 inside the container. Compose publishes that container port to host port 8000 so the browser can call the API using localhost:8000.

5. MediaMTX Dockerfile

Instead of copying the Windows mediamtx.exe into a Linux container, the image downloads the Linux AMD64 release during the image build and copies the existing MediaMTX configuration into the image.

```
FROM alpine:3.20

ARG MTX_VERSION=1.21.0

RUN apk add --no-cache wget ca-certificates && \
    wget -q https://github.com/bluenviron/mediamtx/releases/download/v${MTX_VERSION}/mediamtx_v${MTX_VERSION}_linux_amd64.tar
    tar -xzf mediamtx_v${MTX_VERSION}_linux_amd64.tar.gz && \
    mv mediamtx /usr/local/bin/mediamtx && \
    rm mediamtx_v${MTX_VERSION}_linux_amd64.tar.gz

COPY streaming/mediamtx_v1.21.0_windows_amd64/mediamtx.yml /mediamtx.yml

CMD ["/usr/local/bin/mediamtx", "/mediamtx.yml"]
```

The important portability idea is that the source repository contains the configuration, while the image obtains the correct Linux binary. The Windows executable is not required by the MediaMTX container.

6. Docker Compose File

Docker Compose became the central deployment definition. It created the database, backend and MediaMTX services on the same Docker network.

```
services:

  database:
    image: mysql:8.0
    container_name: gujarat_police_database
    restart: unless-stopped

    environment:
      MYSQL_DATABASE: gujarat_police
      MYSQL_ROOT_PASSWORD: root
```

```
    volumes:
      - mysql_data:/var/lib/mysql
      - ../database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

  backend:
    build:
      context: ..
      dockerfile: deployment/docker/backend.Dockerfile

    container_name: gujarat_police_backend
    restart: unless-stopped

    depends_on:
      - database

    environment:
      DB_HOST: database
      DB_NAME: gujarat_police
      DB_USER: root
      DB_PASSWORD: root

    ports:
      - "8000:8000"

  mediamtx:
    build:
      context: ..
      dockerfile: deployment/docker/mediamtx.Dockerfile

    container_name: gujarat_police_mediamtx
    restart: unless-stopped

    ports:
      - "8554:8554"
      - "8000:8000/udp"
      - "8001:8001/udp"
      - "1935:1935"
      - "8888:8888"
      - "8889:8889"
      - "8189:8189/udp"
      - "8890:8890/udp"
      - "8892:8892"
      - "8892:8892/udp"
      - "8893:8893/udp"
      - "9997:9997"

volumes:
  mysql_data:
```

Note: The exact project file is the source of truth. This document records the working configuration used during development; final production deployments should review secrets and exposed ports.

7. Docker Build Context - A Major Concept

One of the early problems was understanding why the Compose file needed **context: ..**. The Compose file is under **deployment/**, while the Dockerfiles need access to **backend/** and **streaming/** from the project root.

```
Project root
|
+-- backend/
+-- streaming/
+-- database/
+-- deployment/
    +-- docker-compose.yml
    +-- docker/
        +-- backend.Dockerfile
        +-- mediamtx.Dockerfile

From deployment/docker-compose.yml:
context: ..

.. means: project root
```

The Docker build context defines which files Docker is allowed to access during the build. The Dockerfile path is then resolved within that context.

8. Docker Networking - The Key Idea

The backend and MySQL do not need to communicate through Windows. Docker creates an internal network for the Compose services.

```
Docker internal network:

+-------------------------------+
| deployment_default            |
|                               |
| backend                       |
|    |                          |
|    | database:3306            |
|    v                          |
| database                      |
|                               |
| backend                       |
|    |                          |
|    | mediamtx:9997            |
|    v                          |
| mediamtx                      |
+-------------------------------+
```

This means the backend uses **database:3306** for MySQL and **gujarat_police_mediamtx:9997** for MediaMTX API communication. Using 127.0.0.1 from the backend would point back to the backend container itself.

9. Host Port vs Container Port

A mapping such as **3307:3306** means host port 3307 forwards to container port 3306. It is only needed when something outside the Docker network needs direct access to the database.

```
HOST : CONTAINER

3307 : 3306
  |      |
  |      +-- MySQL inside container
  +--------- Windows host port
```

For the final application, the backend already lives inside Docker and can reach MySQL internally, so MySQL does not need a published host port merely for the backend to work. We therefore removed the conflicting 3306 host mapping.

10. Problems Faced and How They Were Solved

MySQL port 3306 was already occupied

Problem: Starting Compose failed because Windows reported that host port 3306 could not be bound. An existing Windows MySQL service was already listening on that port.

Solution: We initially considered changing to 3307, then recognized that the backend does not need host access to MySQL. The host-side MySQL port mapping was removed.

Result: Backend and MySQL communicate over the Docker network using database:3306. The conflict disappeared.

backend.Dockerfile was empty

Problem: Docker could not build the backend image because the Dockerfile contained no build instructions.

Solution: Added the Python base image, working directory, requirements installation, source copy, FFmpeg package and Uvicorn startup command.

Result: The command docker compose build backend completed successfully.

Compose services were inserted at the wrong YAML indentation

Problem: Docker Compose treated mediamtx fields as if they were inside volumes.mysql_data and reported additional properties not allowed.

Solution: Reorganized the YAML so database, backend and mediamtx are siblings under services, while volumes.mysql_data is at the top-level volumes section.

Result: docker compose config became valid.

MediaMTX Dockerfile was empty

Problem: The MediaMTX container had no build instructions.

Solution: Created a Dockerfile that downloads the Linux MediaMTX binary and copies the working configuration into the image.

Result: The MediaMTX image built successfully.

MediaMTX API not reachable on host

Problem: Port 9997 was enabled inside MediaMTX but was initially not published from the container.

Solution: Added 9997:9997 to the MediaMTX ports.

Result: PowerShell could reach the MediaMTX API.

MediaMTX API authentication error

Problem: The API returned authentication error even though the port was reachable.

Solution: Enabled the API and configured a development API user with api permission, then sent HTTP Basic Authentication credentials.

Result: The API returned HTTP 200.

Backend used the wrong MediaMTX address

Problem: Inside Docker, 127.0.0.1:9997 means the backend container itself, not MediaMTX.

Solution: Changed the backend API address to http://gujarat_police_mediamtx:9997.

Result: A test executed from inside the backend container returned HTTP 200 from MediaMTX.

MySQL database existed but tables did not

Problem: gujarat_police existed, but vendor and camera tables were missing because init.sql had not been executed against the already-created volume.

Solution: Executed database/init.sql against the existing database rather than destroying the volume.

Result: SHOW TABLES then displayed vendor and camera.

PowerShell did not accept '<' input redirection

Problem: A CMD/Linux-style command using '< database/init.sql' produced a PowerShell parser error.

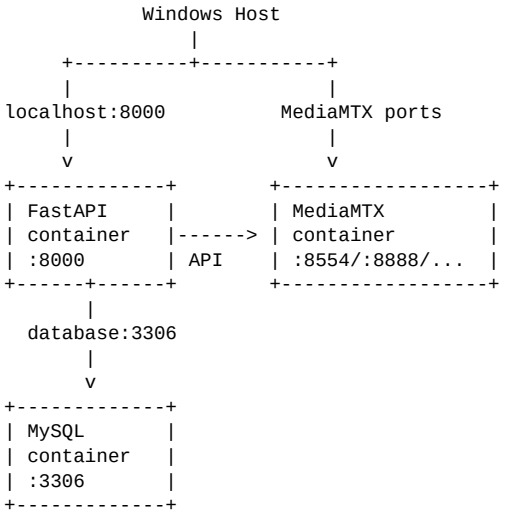
Solution: Used PowerShell-compatible piping/execution to feed the SQL to MySQL.

Result: The schema was initialized successfully.

11. Verification Commands

```
# Validate Compose syntax
docker compose -f .\deployment\docker-compose.yml config

# Build all images and start all services
docker compose -f .\deployment\docker-compose.yml up -d --build

# Show running containers
docker ps

# Show MediaMTX logs
docker logs gujarat_police_mediamtx --tail 50

# Show backend logs
docker logs gujarat_police_backend --tail 100

# Check database exists
docker exec -it gujarat_police_database mysql -uroot -proot -e "SHOW DATABASES;"

# Check tables
docker exec -it gujarat_police_database mysql -uroot -proot -e "USE gujarat_police; SHOW TABLES;"

# Test FastAPI
Invoke-WebRequest http://127.0.0.1:8000/ -UseBasicParsing

# Test MediaMTX API from the host
Invoke-WebRequest http://127.0.0.1:9997/v3/paths/list `
  -Headers @{ Authorization = "Basic <BASE64>" } -UseBasicParsing

# Test MediaMTX API from inside the backend container
docker exec gujarat_police_backend python -c "import httpx; r=httpx.get('http://gujarat_police_mediamtx:9997/v3/paths/list',
```

12. Final Verified Docker Architecture



The real camera was successfully connected to MediaMTX and the browser could view the MediaMTX HLS stream. The FastAPI backend could also reach the MediaMTX Control API through Docker networking.

13. Portability Rules Learned

• Containers communicate using Docker service names and internal ports.

• Host port publishing is only needed when the host or an external client needs access.

• Docker build context must include every file referenced by COPY.

• Configuration files can stay in the repository while machine-specific binaries remain outside it.

• Database volumes persist data across container restarts, so initialization scripts are not automatically rerun on every start.

• Secrets such as camera credentials should not be hard--coded into a public repository.

14. Connection to the MediaMTX + FFmpeg Document

This Docker document is the container/deployment layer for the streaming work documented in the separate MediaMTX + FFmpeg PDF. Together, the two documents describe the progression from local Windows executables to a portable containerized streaming architecture.

```
Part 1 - MediaMTX + FFmpeg
  Camera / test source
        |
        v
    MediaMTX
        |
        +--> RTSP
        +--> HLS
        +--> WebRTC
        +--> Control API

Part 2 - Docker
  MediaMTX -> container
  FastAPI  -> container
  MySQL    -> container
  Docker Compose -> connects all services
```

The next stage can document the Git cleanup and repository workflow, followed by further containerization of FFmpeg where it is actually needed.